

# Render.com Deployment Guide

---

This guide explains how to deploy the Bohloko Family Farm backend to Render.com.

## Prerequisites

- GitHub repository: <https://github.com/Mokgatlam/BohlokoPoultryProject.git>
- Render.com account (free tier available at <https://render.com>)
- Admin access to the repository

## Architecture Overview

When deployed to Render.com:

- **Backend API:** Hosted on Render.com web service
- **Frontend:** Served statically by the backend server
- **Database:** NeDB (file-based, persists on Render's filesystem)
- **URL:** <https://bohloko-family-farm-backend.onrender.com>

## Deployment Steps

### 1. Push Deployment Configuration

The following files have been added for Render.com deployment:

- `render.yaml` - Render service configuration
- `backend/routes/health.js` - Health check endpoint
- Updated `backend/server.js` - CORS and health route

Commit and push these files:

```
git add render.yaml backend/routes/health.js backend/server.js
git commit -m "Add Render.com deployment configuration"
git push origin main
```

### 2. Create Render.com Account

1. Go to <https://render.com>
2. Sign up or log in
3. Connect your GitHub account when prompted

### 3. Deploy Backend to Render.com

#### Option A: Using `render.yaml` (Recommended)

1. Go to <https://dashboard.render.com>

2. Click **New** → **Blueprint**
3. Connect your GitHub repository: `Mokgatlam/BohlokoPoultryProject`
4. Render will automatically detect `render.yaml`
5. Review the configuration:
  - **Service name:** `bohloko-family-farm-backend`
  - **Runtime:** Node.js
  - **Plan:** Free
6. Click **Apply**

Render will automatically:

- Install dependencies
- Start the server
- Deploy your backend
- Provide you with a live URL

### Option B: Manual Setup

If `render.yaml` doesn't work:

1. Go to <https://dashboard.render.com>
2. Click **New** → **Web Service**
3. Connect your GitHub repository
4. Configure the service:
  - **Name:** `bohloko-family-farm-backend`
  - **Runtime:** Node.js
  - **Build Command:** `cd backend && npm install`
  - **Start Command:** `cd backend && npm start`
  - **Plan:** Free
5. Add Environment Variables:

```
NODE_ENV=production
JWT_EXPIRES_IN=1h
```

- **JWT\_SECRET:** Click "Generate" to auto-generate a secure secret
6. Click **Create Web Service**

### 4. Configure Environment Variables

After deployment, go to your service dashboard and verify these environment variables are set:

- `NODE_ENV` = `production`
- `PORT` = `5000` (automatically set by Render)
- `JWT_SECRET` = (auto-generated secure key)
- `JWT_EXPIRES_IN` = `1h`
- `BASE_URL` = (your Render service URL, e.g., `https://bohloko-family-farm-backend.onrender.com`)

- `DATABASE_URL` = (automatically set by Render PostgreSQL service)

**Important:**

- The `BASE_URL` is automatically set by render.yaml using the `fromService` reference
- The `DATABASE_URL` is automatically set by Render PostgreSQL service

## 5. Run Database Seeding (Optional)

To populate your production database with initial data:

1. Go to your service dashboard on Render
2. Click **Shell** tab (opens a web terminal)
3. Run the seed command:

```
cd backend && npm run seed
```

## 6. Access Your Live Application

Once deployed, your application will be available at:

```
https://bohloko-family-farm-backend.onrender.com
```

**Test the deployment:**

- Homepage: <https://bohloko-family-farm-backend.onrender.com/>
- Health check: <https://bohloko-family-farm-backend.onrender.com/api/health>
- API endpoint: <https://bohloko-family-farm-backend.onrender.com/api/products>

## Important Notes

### Free Tier Limitations

Render.com's free tier has some limitations:

1. **Service Sleep:** Services sleep after 15 minutes of inactivity
  - First request after sleep takes 30-60 seconds to wake up
  - Subsequent requests are fast
2. **Monthly Limits:**
  - 512 MB RAM
  - Shared CPU
  - 100 GB bandwidth/month
3. **Database Persistence:**

- NeDB files persist on Render's filesystem
- Data survives deployments but not service deletions
- For production, consider upgrading to PostgreSQL

## Upgrading to PostgreSQL (Recommended for Production)

To use PostgreSQL instead of NeDB:

1. In Render dashboard, click **New** → **PostgreSQL**
2. Configure:
  - **Name:** `bohloko-db`
  - **Database:** `bohloko_farm`
  - **User:** `bohloko_user`
  - **Plan:** Free
3. After creation, get the connection string
4. Update your backend service environment variables:
  - `DATABASE_URL` = (PostgreSQL connection string from Render)
5. Update `backend/config/db.js` to use PostgreSQL instead of NeDB

## CORS Configuration

The backend automatically adds the Render service URL to the CORS whitelist when `NODE_ENV=production` and `BASE_URL` are set.

This allows your frontend to make API requests from the same domain.

## Frontend API Endpoints

The frontend JavaScript (`assets/js/api.js`) must point to your Render backend.

Check if `api.js` has a configurable base URL:

```
// assets/js/api.js
const API_BASE_URL = ''; // Relative URLs work if frontend is served by backend
```

Since the backend serves the frontend statically, relative URLs should work automatically.

## Monitoring and Maintenance

### View Logs

1. Go to your service dashboard on Render
2. Click **Logs** tab
3. View real-time application logs

### Health Monitoring

Render automatically monitors your health check endpoint at `/api/health` and will restart your service if it becomes unresponsive.

## Redeploying

Every time you push to the `main` branch, Render will automatically:

1. Detect the changes
2. Rebuild the application
3. Deploy the new version
4. Update the live URL

Manual redeploys are also available from the dashboard.

## Scaling

If you need more resources:

1. Go to your service settings
2. Change **Plan** from Free to Starter (\$7/month)
3. Benefits:
  - No sleep mode
  - 512 MB RAM (dedicated)
  - Better performance

## Troubleshooting

### Service Won't Start

1. Check the **Logs** tab for errors
2. Common issues:
  - Missing environment variables
  - Port already in use (shouldn't happen on Render)
  - Database connection errors

### CORS Errors

If you see CORS errors:

1. Verify `BASE_URL` environment variable is set correctly
2. Check that `NODE_ENV=production`
3. Verify the frontend is making requests to the correct domain

### NeDB Database Issues

If NeDB doesn't work on Render:

1. Check file permissions in the shell
2. Verify the `data/` directory exists
3. Consider switching to PostgreSQL (see above)

## Slow First Request

This is normal on the free tier. Solutions:

1. Upgrade to paid plan
2. Use a monitoring service to ping your API every 10 minutes
3. Use Render's always-on option (paid)

## Alternative: Full-Stack Deployment

For better performance and no sleep mode:

1. Deploy backend to Render.com (as described above)
2. Deploy frontend to Netlify or Vercel
3. Update frontend API\_BASE\_URL to point to Render backend

This gives you:

- Fast static frontend hosting
- Always-on backend
- Better separation of concerns

## Live URLs

After deployment:

- **Backend API:** <https://bohloko-family-farm-backend.onrender.com>
- **Health Check:** <https://bohloko-family-farm-backend.onrender.com/api/health>
- **Frontend:** <https://bohloko-family-farm-backend.onrender.com/pages/public/index.html>

## Next Steps

1. Sign up at <https://render.com>
2. Push the deployment files to GitHub
3. Create a new Web Service and connect your repo
4. Wait for deployment to complete (2-5 minutes)
5. Visit your live URL
6. Test the API endpoints

For production use, consider:

- Upgrading to a paid plan
- Switching to PostgreSQL
- Adding a custom domain
- Setting up monitoring and alerts